



Khulna University of Engineering & Technology

CSE – 1205 :
OBJECT ORIENTED PROGRAMMING

PHARMACY MANAGEMENT SYSTEM

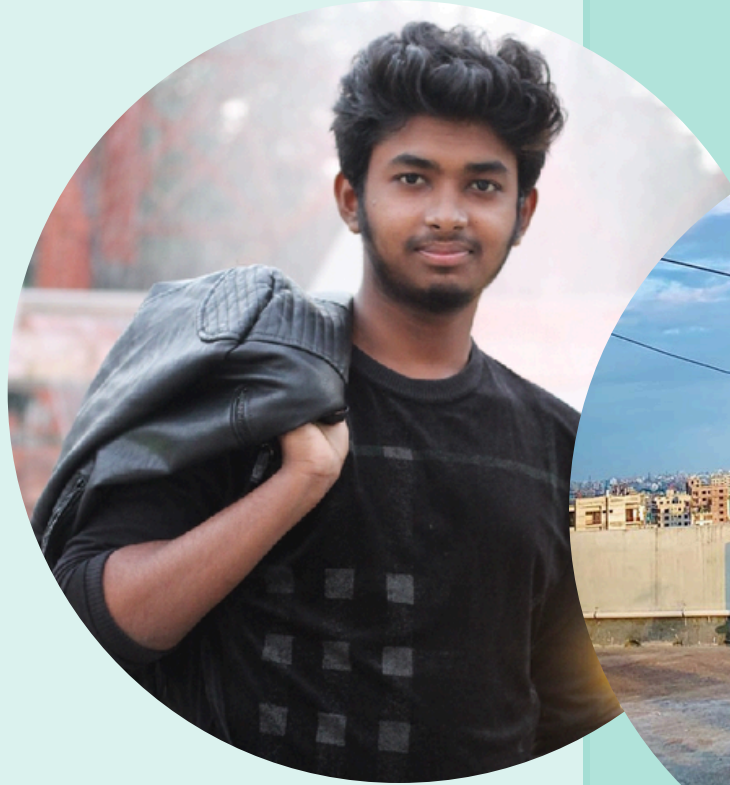
Instructed by:

Safin Ahmed Dipto

Lecturer, Department of CSE,
Khulna University of Engineering & Technology

TEAM

2307085



KRISHNO
MONDOL

2307087



MD. ATIF
ABSAR

2307088

AGOMON
DAS DHRUBO



RITOVASH
CHANDA

ARNOB
DAM



2307091

2307090



RAHANUMA
RASHID **2307089**

System Introduction

The Pharmacy Management System is a comprehensive console-based application designed to streamline pharmacy operations including inventory management, sales tracking, and customer relationship management using Object-Oriented Programming principles.

Key Features

- Inventory Management: Add, update, remove medicines
- Sales Operations: Process sales with invoicing
- Customer Loyalty: Automated discount system
- Expiry Tracking: Alert system for expired medicines
- Stock Management: Low stock warnings
- Reports: Sales reports and analytics
- User Roles: Admin and Pharmacist access levels
- Data Persistence: File-based storage system

OVERVIEW

Header Files Used

```
#include<iostream>
#include<string>
#include<algorithm>
#include<ctime>
#include<sstream>
#include<iomanip>
#include<vector>
#include<fstream>
```

Technologies Used

- Language: C++
- Paradigm: Object-Oriented Programming
- STL: Vectors, Pairs, Algorithms
- File I/O: Text-based persistence

File Structure

- medicines.txt : Stores all medicine inventory data
- customers.txt : Stores customer information and purchase history
- invoices.txt : Records all sales transactions
- invoice_counter.txt : Maintains sequential invoice numbering

PROJECT BREAKDOWN

7 Class Structure -

1. **Medicine Class**: Core inventory entity with operator overloading
2. **Customer Class**: Customer management with loyalty program
3. **Invoice Class** : Billing and transaction management
4. **User Class** : Base class for authentication (Inheritance)
5. **Admin Class** : Complete system management (inherits User)
6. **Pharmacist Class** : Sales operations (inherits User)
7. **PharmacySystem Class** : Main controller and orchestrator

Medicine Class

CLASS

```
int medID;  
string name;  
string company;  
double price;  
int quantity;  
string expiryDate;
```

Key Methods

- updateStock()
- addStock()
- isLowStock()
- isExpired()
- display()

• Operator Overloading

- <> operator :

```
bool  
operator<(const  
Medicine&  
other) const {  
    return price <  
    other.price; }  
}
```

- + operator

```
Medicine  
operator+  
(const  
Medicine&  
other) const {  
    if (medID ==  
    other.medID) {  
        Medicine temp =  
        *this;  
        temp.quantity  
        +=  
        other.quantity;  
        return temp; }  
}
```

• List Serialization

```
// Serialize the medicine list  
string serialize() const {  
    stringstream ss; ss << medID << "," << name << "," <<  
    company << "," << price << "," << quantity << "," <<  
    expiryDate; return ss.str(); }  
}
```

```
ADMIN MENU  
  
1. Add Medicine  
2. Remove Medicine  
3. Update Medicine  
4. View Inventory  
5. View Reports  
6. Check Expiry & Low Stock Alerts  
7. Sort Medicines by Price  
8. Restock Medicine  
9. View Customer Loyalty Ranking  
10. Logout  
Enter choice: 6  
  
EXPIRY & STOCK ALERTS  
  
EXPIRED MEDICINES:  
-----  
No expired medicines.  
  
LOW STOCK MEDICINES (< 10 units):  
-----  
No low stock items.
```

User Class

CLASS

```
int ID;  
string name;  
string contact;  
bool loggedin;
```

Key Methods -

- login(userId, password)
- logout()
- getId()
- getName()
- getContact()
- isLoggedIn()

} Getters

```
WELCOME TO PHARMACY MANAGEMENT SYSTEM  
  
PHARMACY MANAGEMENT SYSTEM  
1. Admin Login  
2. Pharmacist Login  
3. Exit  
Enter choice: 1  
  
--- Admin Login ---  
Enter Admin ID: 23070  
Enter Password: CSE_23  
  
✓ Login successful! Welcome, CSE_23
```

```
10. Logout  
Enter choice: 10  
  
✓ Logged out successfully!  
  
PHARMACY MANAGEMENT SYSTEM  
1. Admin Login  
2. Pharmacist Login  
3. Exit  
Enter choice: █
```

• Login Implementation

```
bool login(int userId,  
const string& password) {  
    if (userId == ID) {  
        loggedin = true; cout <<  
            "\n✓ Login successful!" <<  
            "Welcome, " << name;  
        return true; } return  
        false; }
```

• Logout Implementation

```
void logout() { loggedin =  
false; cout << "\n✓ Logged  
out!"; }
```

• Purpose of User Class

User is an base class that provides common authentication and user management functionality. Both Admin and Pharmacist classes inherit from User, demonstrating the OOP principle of Inheritance.

• Why Use Inheritance?

1 Code Reusability

Login/logout logic written once, used by both Admin and Pharmacist

3 Polymorphism

Can handle Admin and Pharmacist objects as User pointers

2 Protected Access

Member variables accessible to derived classes

4 Extensibility

Easy to add new user roles (Manager, Cashier, etc.)

Admin Class

CLASS

```
vector<Medicine>*  
medicineInventory;  
vector<Customer>*  
customerList;
```

• Inventory Management

- addMedicine()
- removeMedicine()
- updateMedicine()
- viewInventory()
- restockMedicine()
- saveMedicinesToFile()

• Reports & Analytics

- viewReports()
- checkAlerts()
- sortMedicinesByPrice()
- viewCustomerLoyaltyRanking()

MEDICINE INVENTORY						
ID	Name	Company	Price	Quantity	Expiry	Status
1001	Abecma	Bristol Myers Squibb		2.50	2299	27/10/2030
1002	Napa	Beximco Pharmaceuticals Ltd.		1.00	1300	27/10/2030
1003	Napa Extend	Beximco Pharmaceuticals Ltd.		1.50	1200	27/10/2030
1004	Napa Extra	Beximco Pharmaceuticals Ltd.		1.80	1098	27/10/2030
1005	Napa One	Beximco Pharmaceuticals Ltd.		1.80	1200	27/10/2030
1006	Napa Rapid	Beximco Pharmaceuticals Ltd.		1.80	1199	27/10/2030
1007	NapaDol	Beximco Pharmaceuticals Ltd.		1.80	1200	27/10/2030
1008	Ciprofloxacin	Square Pharmaceuticals		5.00	1200	27/10/2030
1009	Doxycycline	Square Pharmaceuticals		5.50	1200	27/10/2030
1010	Entacyd	Square Pharmaceuticals PLC		2.00	1200	27/10/2030
1011	Lactameal	Beximco Pharmaceuticals Ltd.		2.00	1199	27/10/2030
1012	Marlox Plus	Incepta Pharmaceuticals Ltd.		2.00	1200	27/10/2030
1013	Xelosil	Everest Pharmaceuticals Ltd.		2.00	1200	27/10/2030
1014	Tylenol	Johnson & Johnson		5.00	1400	27/10/2030
1015	Panadol	GlaxoSmithKline		5.00	1400	27/10/2030
1016	Ace	Square Pharmaceuticals		1.00	1498	27/10/2030
1017	Bayer Aspirin	Bayer		3.00	2000	27/10/2030
Total Medicines: 17						

• Admin Menu (10 Options)

```
void display(){ cout << " ADMIN MENU " << endl; cout << "1.  
Add Medicine" << endl; cout << "2. Remove Medicine" << endl;  
cout << "3. Update Medicine" << endl; cout << "4. View  
Inventory" << endl; cout << "5. View Reports" << endl; cout  
<< "6. Check Expiry & Low Stock Alerts" << endl; cout << "7.  
Sort Medicines by Price" << endl; cout << "8. Restock  
Medicine" << endl; cout << "9. View Customer Loyalty  
Ranking" << endl; cout << "10. Logout" << endl; }
```

Pharmacist Class

CLASS

```
vector<Medicine>*  
medicineInventory;  
vector<Customer>*  
customerList;  
int invoiceCounter;
```

• Core Functions

- sellMedicine()
- searchMedicine()
- viewInventory()
- checkExpiry()
- saveMedicinesToFile()
- saveCustomersToFile()

• Sales Workflow

- Customer identification
- Search by phone number
- Search medicines
- Create invoice
- Check expiry dates
- Validate stock
- Update inventory
- Print & save invoice

• Pharmacist Menu (5 Options)

```
void showMenu() { cout << " PHARMACIST MENU " << endl; cout  
<< "1. Sell Medicine" << endl; cout << "2. Search Medicine"  
<< endl; cout << "3. View Inventory" << endl; cout << "4.  
Check Expiry" << endl; cout << "5. Logout" << endl; }
```

```
--- Pharmacist Login ---  
Enter Pharmacist ID: 2307090  
Enter Password: Ritovash Chanda  
  
✓ Login successful! Welcome, Ritovash Chanda
```

PHARMACIST MENU

```
1. Sell Medicine  
2. Search Medicine  
3. View Inventory  
4. Check Expiry  
5. Logout  
Enter choice: 1
```

```
--- Sell Medicine ---  
Is this an existing customer? (y/n): n  
Enter Customer Name: Rahanuma Rashid  
Enter Customer Phone: 017XXXXXXX
```

```
Enter Medicine ID: 1003  
Medicine: Napa Extend  
Available Quantity: 1200  
Price: $1.50  
Enter Quantity to sell: 50  
✓ Added to invoice!
```

```
Add more medicines? (y/n): n
```

Invoice Class

CLASS

Key Methods

- addItem(Medicine, qty)
- applyDiscount(percent)
- generateBill(discount)
- printInvoice()
- saveToFile()
- getFinalAmount()

• Adding Items

```
// Adding item to invoice
void addItem(const Medicine&
med, int qty) {
    items.push_back(make_pair(med,
qty)); totalAmount +=
med.getPrice() * qty; }
```

PHARMACY MANAGEMENT SYSTEM INVOICE			
Invoice ID: 1005 Date: 28/10/2025 Customer: agomon			
Medicine	Price	Quantity	Subtotal
Tylenol	5.00	5	25.00
			Total: \$25.00
			Final Amount: \$25.00
Thank you for your purchase!			
✓ Sale completed successfully!			

```
int invoiceID;
string customerName;
string date;
vector<pair<Medicine,
int>> items;
double totalAmount;
double
discountAmount;
double finalAmount;
```

• Discount & Bill Generation

```
// Discount and generate bill void applyDiscount(double
discountPercent) { discountAmount = totalAmount *
(discountPercent / 100.0); finalAmount = totalAmount -
discountAmount; } void generateBill(double discountPercent =
0.0) { applyDiscount(discountPercent); }
```

• File Storage

```
// Saving invoice to a file void saveToFile() const{
ofstream file("invoices.txt", ios::app); if(file.is_open()){
file << invoiceID << "," << customerName << "," << date <<
"," << totalAmount << "," << discountAmount << "," <<
finalAmount << endl; file.close(); } }
```


Customer Class

CLASS

```
int custID;  
string name;  
string phone;  
int purchaseCount;
```

- **Display**

- Customer ID
- Name
- Phone
- Total Purchases

- **Discount Percentage**

```
// Discount and purchasecount method  
void incrementPurchase(){  
    ++(*this);  
}  
bool isEligibleForDiscount()const{  
    return purchaseCount >= 5;  
}  
double getDiscountPercentage()const{  
    if (purchaseCount >= 10) return 10.0;  
    if (purchaseCount >= 5) return 5.0;  
    return 0.0;  
}
```

- 5% discount :
5+ purchases
- 10% discount :
10+ purchases

- **Operator Overloading**

- ++ operator :

```
Customer& operator++(){  
    purchaseCount++;  
    return *this;  
}
```

- < operator

```
// Overloaded < operator for customer class  
bool operator<(const Customer& other)const{  
    return purchaseCount < other.purchaseCount;  
}
```

- **List Serialization/ Deserialization**

```
// Serialize the customer lists  
string serialize() const {  
    stringstream ss;  
    ss << custID << "," << name << "," << phone << "," << purchaseCount;  
    return ss.str();  
}
```

CUSTOMER LOYALTY RANKING					
Rank	Cust ID	Name	Phone	Purchases	Discount
1	7	Atif Absar	01759977867	1	0.00%
2	6	agomon	00000000	1	0.00%
3	5	Arnob Dam	01733849923	1	0.00%
4	4	Arnob Saha	01534487623	1	0.00%
5	3	Aditya Banik	01534889333	1	0.00%
6	2	Md. Farjamul Islam	01712345578	1	0.00%
7	1	GGGG	098888	0	0.00%
Total Customers: 7					

PharmacySystem Class

Main Control Methods

- displayMainMenu()
- adminLogin()
- pharmacistLogin()
- adminMenu()
- pharmacistMenu()
- run()

• System Initialization

```
// Constructor PharmacySystem() { // Initializing Users admin
= new Admin(23070, "CSE_23", "015", &medicineInventory,
&customerList); pharmacist = new Pharmacist(2307090,
"Ritovash Chanda", "01615058161", &medicineInventory,
&customerList); loadMedicinesFromFile();
loadCustomersFromFile(); }
```

```
--- Admin Login ---
Enter Admin ID: 23070
Enter Password: CSE_23
```

```
ADMIN MENU

1. Add Medicine
2. Remove Medicine
3. Update Medicine
4. View Inventory
5. View Reports
6. Check Expiry & Low Stock Alerts
7. Sort Medicines by Price
8. Restock Medicine
9. View Customer Loyalty Ranking
10. Logout
Enter choice: 6
```

```
--- Pharmacist Login ---
Enter Pharmacist ID: 2307090
Enter Password: Ritovash Chanda
```

```
PHARMACIST MENU

1. Sell Medicine
2. Search Medicine
3. View Inventory
4. Check Expiry
5. Logout
Enter choice: 1
```

CLASS

```
vector<Medicine>
medicineInventory;
vector<Customer>
customerList;
Admin* admin;
Pharmacist*
pharmacist;
```

• Data Persistence

- loadMedicinesFromFile()
- loadCustomersFromFile()

```
// Load Medicines Info from file
void loadMedicinesFromFile() {
ifstream file("medicines.txt");
if (file.is_open()) { string
line; while (getline(file,
line)) { Medicine med =
Medicine::deserialize(line); if
(med.getMedID() != 0) {
medicineInventory.push_back(med);
} } file.close(); } }
```

Conclusion & Topics Covered

OOP Concepts Implemented

- Encapsulation: Private data members with public getters/setters
- Inheritance: User → Admin & Pharmacist
- Polymorphism: Operator Overloading
- Abstraction: Interface separation for different user roles

Advanced C++ Features

- Operator Overloading: +, ++, <, >, ==
- STL Containers: vector, pair
- File I/O: Serialization/Deserialization
- Lambda Functions: Used in sorting algorithms
- Pointers: Dynamic memory management
- Templates: Generic programming with STL

• Key Achievements

✓ Successfully implemented all core OOP principles ✓ Created a fully functional pharmacy management system ✓ Efficient file-based data persistence ✓ User-friendly console interface with formatted output ✓ Robust error handling and validation ✓ Automatic discount calculation based on loyalty ✓ Real-time inventory tracking with alerts ✓ Professional invoice generation



ANY QUESTIONS?

PLEASE GO AHEAD



THANK YOU